EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 9-IDA: IDA Pro — Read, Hack, and Patch Operator Immediates and a DHT11 Float Scale (Raw `.bin`)

ARM Cortex-M33 • GDB • Ghidra

October 17, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build the lesson project with `Release` and get both an `.elf` and a raw `.bin`
- Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
- Load the raw `.bin` into IDA at `0x10000000`
- Read the six C operator results as **bare instruction immediates** (`movs r1, #50` , `movs r1, #5` , …)
- Read the DHT11 single-wire driver, including the inlined `gpio_*` and `time_us_32` helpers ted value live t
- **Resolve the functions in the IDA GUI** using the ELF symbol map, including the `dht11.c` symbols
- **Patch** the arithmetic immediate (`50 -> 99`) and the DHT11 scaling constant (`0.1f -> 5.0f`)
- **Rename** the printed label by patching the `"arithmetic_operator:"` string
- **Export** the patched image, convert it to UF2, and export it

How This Guide Works

The build produces two files for the project:

File	What it is	How we use it
`.elf`	The linked image with a full symbol table	Ground truth for every function address and name
`.bin`	The raw flash image, no headers, no symbols	The image we load into IDA and reverse

The `.bin` is built **from** the `.elf` , so the ELF tells you exactly what is at every address. We use the ELF symbol map to resolve functions in IDA, and we reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

> **Build `Release`, not `Debug`.** Every address in this guide matches the Week 9 lesson, and the Week 9 lesson is a `Release` build. `Release` optimizes the code the same way the original lesson was built: it **inlines** the `static` helpers `print_operator_results`, `compute_arithmetic_ops`, `compute_operators`, and `print_dht11_reading` straight into `main`, and it inlines the whole `dht11.c` static helper chain (`send_start_signal`, `wait_for_level`, `wait_response`, `read_bit`, `read_40_bits`, `validate_checksum`) into `dht11_read`. Every operator result is folded to a bare `movs r1, #imm` immediate. If you build `Debug`, the SDK function addresses move and the helpers stay separate calls, so nothing lines up. Always build `Release` for this lesson.

The order is **dynamic first, static second**:

d prove what the code does. ge. 3. Resolve the functions in IDA using the ELF symbol map. 4. Patch the bytes, export, convert and export.

Project	Serial output	Also does	The hacks
`0x001a_operators`	six operator lines, then `Humidity: …%, Temperature: …°C`	DHT11 single-wire sensor on GPIO 4	`50 -> 99` (arithmetic immediate), `0.1f -> 5.0f` (DHT11 scale), and `"arithmetic_operator:" -> "hacked_operator:"`

> **Addresses come from your build.** Every address here is from the `Release` build produced in Step 3. Confirm against your own `.elf` with the command in Step 4.

> **The surprise of this week is how small the operator code is.** The C source computes `x * y`, `x > y`, `x << 1`, and `x += 5` in a helper function, but `Release` folds every one of those into a compile-time constant and stores it as a one-byte immediate inside a `movs r1, #imm` instruction. There is no variable in memory to change — the answer is baked into the instruction stream. The DHT11 reading is different: it is a real measurement scaled by a `float` constant (`0.1f`) sitting in the literal pool, and *that* one is a data word you can patch.

> **A note on the sensor reading.** The humidity/temperature numbers depend on the physical DHT11 attached to GPIO 4. This guide's ground truth is about the *code*, not the sensor: the scale constant, where `dht11_read` lives, and which register holds the reading.

Part 1: Build and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **IDA Pro** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

bash

````
**Windows x64** and **Linux x64** do not have this problem. Skip to Step 3.

### Step 3: Build the project with `Release`

Run this inside `0x001a_operators/`:

```bash
cmake -B build -G Ninja -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 -DCMAKE_BUILD_TYPE=Release
cmake --build build
````

Point IDA at this repository (once). Every console snippet below reads the repo root from `~/.embedded-hacking-repo`, so IDA never needs a database open and nothing is hardcoded. From the repo root, run once:

macOS / Linux:

```
pwd > ~/.embedded-hacking-repo
```

Windows (PowerShell):

```
(Get-Location).Path | Set-Content "$env:USERPROFILE\.embedded-hacking-repo"
```

Then build from the IDA console, so the whole build -> patch -> flash loop stays inside IDA. The console inherits a minimal `PATH` — on macOS just `/usr/bin:/bin:/usr/sbin:/sbin` — so it does not see Homebrew; add your package manager's `bin` first, then run plain `cmake`.

macOS Apple Silicon:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
os.environ["PATH"] = "/opt/homebrew/bin:" + os.environ["PATH"]  # the console's PATH omits Homebrew
proj = os.path.join(root, "0x001a_operators")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x001a_operators")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
proj = os.path.join(root, "0x001a_operators")
subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
subprocess.run(["cmake", "--build", "build"], cwd=proj)
```
```

The build directory now contains the pair we need:

- `0x001a_operators/build/0x001a_operators.elf` and `.bin` — the `.bin` is **17484** bytes (`0x444c`)

If the ARM toolchain is not on your `PATH`, add `-DPICO_TOOLCHAIN_PATH=...`:

OS	Typical toolchain path
Windows x64	`C:/Program Files/Arm GNU Toolchain arm-none-eabi/14.2 rel1/bin`
macOS Apple Silicon	`~/.pico-sdk/toolchain/14_2_Rel1/bin`
Linux x64	`/usr`

> **This guide's toolchain lives at** `~/.pico-sdk/toolchain/14_2_Rel1/bin`**.** All of `arm-none-eabi-nm`, `arm-none-eabi-objdump`, and `arm-none-eabi-gdb` resolved in this document come from there. If your install is elsewhere, `which arm-none-eabi-nm` tells you where to point.

Step 4: Dump the ELF symbol map

This is the ground truth for the whole lesson. Run `arm-none-eabi-nm` on the ELF and keep the output in a terminal or a text file:

macOS Apple Silicon / Linux x64:

```
arm-none-eabi-nm -n --defined-only build/0x001a_operators.elf | grep -E ' [Tt] '
```

Windows x64:

```
arm-none-eabi-nm -n --defined-only build\0x001a_operators.elf | Select-String ' [Tt] '
```

Each line is `address type name`. The `T` / `t` type is a function. Here are the functions this lesson uses. The signatures come from the ELF's DWARF debug info queried with `arm-none-eabi-gdb -batch -ex "ptype <name>"`, so they are exact.

Our code and the startup chain:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init`, `main`, `exit`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function (all four static helpers inlined)
`0x100002d4`	`dht11_init`	`void dht11_init(uint8_t)`	the `dht11.c` init function

Address	ELF symbol	Signature	Role
`0x100002f4`	`dht11_read`	`bool dht11_read(float*, float*)`	the `dht11.c` read function (all six static helpers inlined)

The GPIO and timer functions `main` / `dht11.c` reach:

Address	ELF symbol	Signature	Role
`0x10000440`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO function select (UART pins)
`0x1000047c`	`gpio_set_pulls`	`void gpio_set_pulls(uint, bool, bool)`	SDK pull config (`gpio_pull_up` inlined)
`0x100004a4`	`gpio_init`	`void gpio_init(uint)`	SDK GPIO init (`dht11_init` calls it)
`0x10000f00`	`sleep_us`	`void sleep_us(uint64_t)`	SDK microsecond delay (`dht11_read` calls it)
`0x10000fd8`	`sleep_ms`	`void sleep_ms(uint32_t)`	SDK millisecond delay (`main`, `dht11_read`)
`0x100011bc`	`time_us_64`	`uint64_t time_us_64(void)`	SDK microsecond clock (`printf` path)
`0x100011d0`	`busy_wait_us`	`void busy_wait_us(uint64_t)`	UART timing loop
`0x10001250`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`	SDK UART init
`0x10001424`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`	UART clock lookup

The stdio/UART and printf chain `main` reaches:

Address	ELF symbol	Signature	Role
`0x1000313c`	`exit`	`void exit(int)`	C runtime exit
`0x10003144`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x10003170`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`	CRLF output driver
`0x10003280`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`	buffered string output
`0x1000336c`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`	enable the UART driver
`0x10003394`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init

Address	ELF symbol	Signature	Role
`0x10003424`	`__wrap_puts`	`int __wrap_puts(const char*)`	the `puts` wrapper (`DHT11 read failed`)
`0x10003460`	`__wrap_vprintf`	`int __wrap_vprintf(const char*, va_list)`	printf core
`0x10003524`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`	the `printf` wrapper (all seven calls)
`0x100036e0`	`stdio_uart_init`	`void stdio_uart_init(void)`	SDK UART stdio init
`0x10003820`	`strlen`	`size_t strlen(const char*)`	C runtime string length

SDK helpers the `printf` path reaches:

Address	ELF symbol	Signature	Role
`0x10001a44`	`_out_rev`	`unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)`	reversed-digit output
`0x10001ae0`	`_ntoa_format`	`unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)`	number formatter
`0x10001cb4`	`_out_char`	`void _out_char(char, void*, size_t, size_t)`	single-char sink
`0x100026fc`	`_vsnprintf`	`int _vsnprintf(out_fct_type, char*, size_t, const char*, va_list)`	the format dispatcher
`0x100030e0`	`vfctprintf`	`int vfctprintf(void (*)(char, void*), void*, const char*, va_list)`	printf format engine

Things in this project that have **no symbol of their own**, because the compiler inlined them:

- `print_operator_results`, `compute_arithmetic_ops`, `compute_operators`, `print_dht11_reading` — the four `static` helpers in `0x001a_operators.c` are inlined into `main`, so their bodies appear directly inside `main`.
- `send_start_signal`, `wait_for_level`, `wait_response`, `read_bit`, `read_40_bits`, `validate_checksum` — the six `static` helpers in `dht11.c` are inlined into `dht11_read`, along with `gpio_set_dir`, `gpio_put`, `gpio_get`, and `time_us_32` from the SDK headers. You see `mcrr` (SIO writes), `ldr [.., #4]` (SIO reads), and `ldr [r0, #40]` (TIMERo read) directly inside `dht11_read`.
- `gpio_pull_up` — `static inline` in the SDK, so `dht11_init` compiles to a direct tail-call to `gpio_set_pulls`.

> `dht_pin` **is a** `b` **symbol, not a function.** `arm-none-eabi-nm -n` lists `20000b7c b dht_pin`. That lowercase `b` is a **local data** symbol: the `dht11.c` file-scope `static uint dht_pin;` is parked in RAM `.bss` at `0x20000b7c`. It holds the GPIO number (`4`) after `dht11_init(4)` runs. There is no function there; do not `Y` it with a prototype.

Part 2: Load the Raw `.bin` into IDA

Start from a fresh IDA state. If you already have a database for this lesson, **close it and start over**; a stale database keeps old names and patches.

Step 7: Bring the raw `.bin` into IDA

A raw `.bin` has no headers, so IDA cannot know where it belongs or what architecture it is. You must supply both. If you just double-click the `.bin`, IDA may load it with a guessed architecture, and every address in this lesson will be wrong.

1.1 Open the file

1. **File ▸ Open...** (Ctrl/Cmd+O).
2. Select the `.bin` file and click **Open**.

IDA opens the **"Load a new file"** dialog because a raw `.bin` has no header.

1.2 Set the format and processor

In the *Load a new file* dialog:

1. **Format**: leave it as **Binary file** (IDA auto-detects a headerless image).
2. **Processor type**: click the **...** button next to the processor field. In the *Processor type* dialog:
   - Family: **ARM**
   - Type: **ARM Little-endian** (the 32-bit ARM module).
   - Click **OK**.

> If IDA asks **"Do you want to disassemble it as 64-bit code?"** — click **No**. The RP2350 is 32-bit; answering *Yes* loads AArch64 and you get `X` registers / data. (This is the single most common cause of the `DCB` wall.)

1.3 Configure the ARM architecture and Thumb mode

Still in the *Load a new file* dialog, open the processor options (the **Processor options** / **Set...** button, which opens the dialog titled **ARM architecture options**):

- **ARM architecture**: choose **ARMv8-M** (this is the Cortex-M33 / RP2350 core).
- **Thumb instructions**: select **Thumb-2** (ARMv8-M is Thumb-only, so this is the correct mode).
- Leave **automatic ARM-THUMB switch** off; we want Thumb throughout.

1.4 Set the loading address

- **Loading address**: enter `10000000` (hex; the RP2350 XIP flash base). IDA rejects addresses that don't look like ROM/RAM with *"The loading address should belong to RAM or ROM"* — `0x10000000` is the correct flash window.

Leave **Manual load** and **Create segments** at their defaults. Click **OK**.

1.5 Force Thumb, then seed the code from the vector table

A raw `.bin` has no header, so IDA's loader has **no entry point** and defaults the segment to **ARM mode** (which cannot decode Thumb). Two required steps — in this order:

A. Set the segment to Thumb (do this first).

- **Edit ▸ Segments ▸ Set default segment register value…** → register `T` = `1` (or press `Alt+G`). IDA reanalyzes the segment as Thumb automatically.

B. Seed the code from the vector table.

1. **G** → `10000000`. The first two words are the initial stack pointer and the **reset vector**.
2. **Read the 4-byte word at** `0x10000004` (do **not** assume its value — it differs per build).
3. **Clear bit 0** (the ARM **Thumb** flag) → the reset handler address.
4. **G** to that address → press **C** (**MakeCode**). IDA disassembles the reset handler and follows its `bl`s into `main`.

Once `main` shows code, the import is correct.

Why IDA needs this and others don't: IDA's generic raw-binary ARM loader treats the image as flat data and waits for you to set Thumb and mark the entry point. It's an IDA quirk, not a mistake on your part.

Step 8: The views you will use

- **Linear view:** the disassembly listing. You navigate, read, and patch here.
- **Graph view:** the control-flow graph of the current function.
- **Decompiler (HLIL):** the pseudo-C decompilation.
- **Hex view:** raw bytes, used for patching.
- **Function list:** the sidebar list of every detected function.

Navigation: `G` go to address, `N` rename, `Y` set type or signature, `;` add a comment.

> **macOS function keys:** the top-row `F` keys are usually mapped to system functions. Every step here uses menu paths that work without them.

> **Optional — load the RP2350 SVD for peripheral names.** If you want IDA to label peripheral registers (TIMER0, SIO, UART0, …) instead of bare addresses, load the RP2350 SVD. It ships with Week 4, so it lives at `WEEK04/rp2350.svd` inside the repository — remember it is in **Week 4**, not this week. This lesson does not need it; the ELF symbol map already names every function.

Part 3: Static — Resolve the Functions in IDA and Patch

Step 16: Resolve the functions in the IDA GUI

We now name the functions in IDA using the ELF symbol map from Step 4. IDA loaded the raw `.bin` with **no symbols**, so every function shows as `sub_<addr>` — resolving means giving each one its real name and signature.

Three keys do all the work:

Key	IDA action	Use it for
`G`	Go to address	Jump to a function's address
`Y`	**Change Type**	Set the function's signature. The dialog shows the full prototype, so this sets the name *and* the type in one step.

Key	IDA action	Use it for
`N`	Rename	Rename only, when you just want the name and not the type

For each function below: `G` to its address, then `Y` **(Change Type)** and type the prototype from the table.

How to resolve a function in IDA (`Y`)

`Y` is the **Change Type** key, and it is what actually resolves the function — it turns `void sub_10003394()` into `bool stdio_init_all(void)`. The Change Type dialog shows the full declaration (name and type), so typing the prototype sets both:

1. `G` to the function's address. The cursor lands on the function.
2. Press `Y`. In the Change Type dialog, type the prototype from the table exactly — for example `bool stdio_init_all(void)` — and press Enter.

The decompiler header then shows the real prototype, and calls to the function read cleanly instead of `sub_<addr>()`. `N` is only for renaming without touching the type; `Y` alone sets both the name and the type.

If `Y` seems to do nothing, confirm the cursor is on the function, or right-click it and pick **Change Type...**. IDA parses what you type and silently keeps the old type if it does not parse, so glance at the header after each `Y`.

Worked example: `main`

1. Press `G`, type `0x10000234`, press Enter. The view jumps there; the cursor lands on `sub_10000234`.
2. Press `Y` (Change Type), type `int main(void)`, press Enter. That sets the name to `main` and the type to `int(void)`.

> **IDA shows `int32_t` where Ghidra shows `int`.** After you set `int main(void)`, the decompiler header may read `int32_t main(void)`. That is the same type — on this platform `int` is 32 bits and IDA's parser normalises it to `int32_t`. Do not fight it; it is not an error.

Worked example: `dht11_init`

1. `G` -> `0x100002d4`.
2. `Y` -> `void dht11_init(uint8_t pin)`.

This is our own `dht11.c` code. In this build the SDK's `gpio_init` and `gpio_pull_up` are called directly (`gpio_pull_up` as a tail-call to `gpio_set_pulls`), so the function is small.

Worked example: `dht11_read`

1. `G` -> `0x100002f4`.
2. `Y` -> `bool dht11_read(float* humidity, float* temperature)`.

This is the big one: all six `dht11.c` static helpers are inlined here, along with `gpio_set_dir` / `gpio_put` / `gpio_get` / `time_us_32`, so `dht11_read` contains the entire single-wire protocol and the `0.1f` scale.

Worked example: `gpio_init`

1. `G` -> `0x100004a4`.
2. `Y` -> `void gpio_init(uint gpio)`.

Worked example: `gpio_set_pulls`

1. `G` -> `0x1000047c`.
2. `Y` -> `void gpio_set_pulls(uint gpio, bool up, bool down)`.

This is what `gpio_pull_up(4)` in `dht11_init` compiles to.

Worked example: `__wrap_printf`

1. `G` -> `0x10003524`.
2. `Y` -> `int __wrap_printf(const char *fmt, ...)`. Keep the `...` — `printf` is variadic.

> **`printf` in our source is `__wrap_printf` in the binary.** The SDK links our `printf` calls to its `__wrap_printf` wrapper, which forwards to `__wrap_vprintf`. Rename it `printf` if you prefer the lesson's shorthand, but `__wrap_printf` is what the ELF says.

Worked example: `__wrap_puts`

1. `G` -> `0x10003424`.
2. `Y` -> `int __wrap_puts(const char *s)`.

This is the `else` branch of `print_dht11_reading`, which prints `"DHT11 read failed\r\n"`. (`__wrap_puts` is aliased to `stdio_puts` at the same address.)

The rest of the chain is the same two keystrokes per function (`G`, then `Y`). This is **our code plus the library functions it actually calls** — not the whole SDK. `main` calls `stdio_init_all`, `dht11_init`, `__wrap_printf`, `dht11_read`, `__wrap_puts`, and `sleep_ms`, so we follow that chain down.

The call chain for this project:

```
main
├── stdio_init_all ── stdio_uart_init ── gpio_set_function, uart_init, stdio_set_driver_enabled
│                                        └── uart_init ── clock_get_hz, busy_wait_us
├── dht11_init ── gpio_init, gpio_set_pulls        (gpio_pull_up inlined)
├── __wrap_printf ── __wrap_vprintf ── vfctprintf ── _vsnprintf
│   │                                                └── _ntoa_format / _out_rev / _out_char
│   ├── stdio_out_chars_crlf
│   └── time_us_64
├── dht11_read ── sleep_ms, sleep_us
│   └── send_start_signal / wait_for_level / wait_response / read_bit /
│       read_40_bits / validate_checksum / gpio_set_dir / gpio_put /
│       gpio_get / time_us_32   (all inlined; no calls)
└── __wrap_puts ── stdio_put_string
```

Resolve every function in that chain:

Address	Rename to (`N`)	Signature (`Y`)
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`
`0x10000186`	`platform_entry`	`void platform_entry(void)`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`
`0x100001e4`	`_init`	`void _init(void)`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`
`0x10000234`	`main`	`int main(void)`

Address	Rename to (N)	Signature (Y)
0x100002d4	dht11_init	void dht11_init(uint8_t)
0x100002f4	dht11_read	bool dht11_read(float*, float*)
0x10000440	gpio_set_function	void gpio_set_function(uint, gpio_function_t)
0x1000047c	gpio_set_pulls	void gpio_set_pulls(uint, bool, bool)
0x100004a4	gpio_init	void gpio_init(uint)
0x10000f00	sleep_us	void sleep_us(uint64_t)
0x10000fd8	sleep_ms	void sleep_ms(uint32_t)
0x100011bc	time_us_64	uint64_t time_us_64(void)
0x100011d0	busy_wait_us	void busy_wait_us(uint64_t)
0x10001250	uart_init	uint uart_init(uart_inst_t*, uint)
0x10001424	clock_get_hz	unsigned long clock_get_hz(clock_handle_t)
0x1000313c	exit	void exit(int)
0x10003144	runtime_init	void runtime_init(void)
0x10003170	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)
0x10003280	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)
0x1000336c	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)
0x10003394	stdio_init_all	bool stdio_init_all(void)
0x10003424	__wrap_puts	int __wrap_puts(const char*)
0x10003460	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)
0x10003524	__wrap_printf	int __wrap_printf(const char*, ...)
0x100036e0	stdio_uart_init	void stdio_uart_init(void)
0x10003820	strlen	size_t strlen(const char*)

Resolve the SDK helpers the `printf` path reaches:

Address	Rename to (N)	Signature (Y)
0x10001a44	_out_rev	unsigned _out_rev(out_fct_type, char*, size_t, size_t, const char*, size_t, unsigned, unsigned)
0x10001ae0	_ntoa_format	unsigned _ntoa_format(out_fct_type, char*, size_t, size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)
0x10001cb4	_out_char	void _out_char(char, void*, size_t, size_t)

Address	Rename to (N)	Signature (Y)
`0x100026fc`	`_vsnprintf`	`int _vsnprintf(out_fct_type, char*, size_t, const char*, va_list)`
`0x100030e0`	`vfctprintf`	`int vfctprintf(void (*)(char, void*), void*, const char*, va_list)`

> **A `void` return type may not stick — here is the fix.** IDA treats `void` as low-confidence, and its analysis can override it with an inferred type — most often `int32_t` on this 32-bit target. It is most visible on `_reset_handler` (a hand-written assembly entry that never returns normally), but it can happen to **any** function whose return type IDA thinks it can infer.
>
> Setting the full signature with `Y` reproduces the unwanted `int32_t`, and `fn.return_type = ...` fails too. What works is the **return-value** setter:
>
> ```
> from binaryninja import ReturnValue, Type
> fn = bv.get_function_at(0x1000015c)
> if fn is not None:
>     fn.return_value = ReturnValue(Type.void())
> ```
>
> That holds `_reset_handler` at `void` even after reanalysis. If it still will not stick, leave it — it does not affect the rest of the lesson.

> `dht_pin` **is data, not a function.** `arm-none-eabi-nm -n` lists `20000b7c b dht_pin`. That lowercase `b` is a **local data** symbol: the linker parks the `dht11.c` file-scope static at RAM address `0x20000b7c`. `dht11_init(4)` stores `4` into it. There is no function there; do not `Y` it with a prototype.

Shortcut — resolves name *and* type for every function. Instead of doing `N` + `Y` by hand, paste this into IDA's Python console (`Plugins -> Python Console`). It sets each function's name and signature programmatically:

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
typedef unsigned long clock_handle_t;
typedef void (*out_fct_type)(char, void*, size_t, size_t);
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature)
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x100002d4: ("dht11_init",                "void dht11_init(uint8_t)"),
    0x100002f4: ("dht11_read",                "bool dht11_read(float*, float*)"),
    0x10000440: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x1000047c: ("gpio_set_pulls",            "void gpio_set_pulls(uint, bool, bool)"),
    0x100004a4: ("gpio_init",                 "void gpio_init(uint)"),
    0x10000f00: ("sleep_us",                  "void sleep_us(uint64_t)"),
    0x10000fd8: ("sleep_ms",                  "void sleep_ms(uint32_t)"),
    0x100011bc: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x100011d0: ("busy_wait_us",              "void busy_wait_us(uint64_t)"),
    0x10001250: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x10001424: ("clock_get_hz",              "unsigned long clock_get_hz(clock_handle_t)"),
    0x1000313c: ("exit",                      "void exit(int)"),
    0x10003144: ("runtime_init",              "void runtime_init(void)"),
    0x10003170: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x10003280: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x1000336c: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x10003394: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x10003424: ("__wrap_puts",               "int __wrap_puts(const char*)"),
    0x10003460: ("__wrap_vprintf",            "int __wrap_vprintf(const char*, va_list)"),
    0x10003524: ("__wrap_printf",             "int __wrap_printf(const char*, ...)"),
    0x100036e0: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x10003820: ("strlen",                    "size_t strlen(const char*)"),
    0x10001a44: ("_out_rev",                  "unsigned _out_rev(out_fct_type, char*, size_t,
size_t, const char*, size_t, unsigned, unsigned)"),
    0x10001ae0: ("_ntoa_format",              "unsigned _ntoa_format(out_fct_type, char*, size_t,
size_t, char*, size_t, bool, unsigned, unsigned, unsigned, unsigned)"),
    0x10001cb4: ("_out_char",                 "void _out_char(char, void*, size_t, size_t)"),
    0x100026fc: ("_vsnprintf",                "int _vsnprintf(out_fct_type, char*, size_t, const
char*, va_list)"),
    0x100030e0: ("vfctprintf",                "int vfctprintf(void (*)(char, void*), void*, const
char*, va_list)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    f = bv.get_function_at(addr)
```

```
    if f is not None:
        f.set_user_type(sig)
```

SDK type names (`stdio_driver_t` , `uart_inst_t` , `gpio_function_t` , plus `uint` , `va_list` , `clock_handle_t` , and `out_fct_type`) are **not** in the raw `.bin` . `set_user_type` re-parses each signature as C, so an undefined name raises `SyntaxError: unknown type name '...'` and stops the loop — it is not harmless. The `sdk` block above defines them first (an opaque `struct` / `enum` / `typedef` is enough to parse). Standard names (`uint8_t` , `uint32_t` , `uint64_t` , `bool` , `size_t`) are built in. If you add a function that uses another SDK type, add a definition for it to that block too.

Step 17: Read `main` in the decompiler

Open the **Decompiler** view on `main` . Once the functions above are typed, it reads roughly:

```
int32_t main(void)
{
    stdio_init_all();
    dht11_init(4);
    while (true) {
        __wrap_printf("arithmetic_operator: %d\r\n", 0x32);   // 50
        __wrap_printf("increment_operator: %d\r\n", 5);       // 5
        __wrap_printf("relational_operator: %d\r\n", 0);      // false
        __wrap_printf("logical_operator: %d\r\n", 0);         // false
        __wrap_printf("bitwise_operator: %d\r\n", 0xa);       // 10
        __wrap_printf("assignment_operator: %d\r\n", 0xa);    // 10
        if (!dht11_read(&hum, &temp)) {
            __wrap_puts("DHT11 read failed\r\n");
        } else {
            __wrap_printf("Humidity: %.1f%%, Temperature: %.1f°C\r\n", hum, temp);
        }
        sleep_ms(2000);
    }
}
```

The `0x32` , `5` , `0` , `0xa` , and `0xa` are the constants we will patch first. Every one is an **immediate in the instruction stream** — there is no variable in memory to change, which is why the patch edits the instruction operand.

Step 18: Patch 1 — change `arithmetic_operator` from 50 to 99

Go to `0x1000026e` :

```
1000026e: 32 21   movs  r1, #50   @ 0x32
```

`movs r1, #imm8` is a 16-bit Thumb instruction. Its encoding is `0x21XX` (`movs r1, #imm8`), stored little-endian as `XX 21` — so the immediate is the byte at the instruction's **own** address, `0x1000026e` . Change `0x32` (50) to `0x63` (99).

> **This is the opposite byte from the Week 7 `movw` patch.** For a 16-bit `movs` , the immediate is the first byte. For the 32-bit `movw` , the low immediate byte was the *third* byte. Always decode the instruction before you decide which byte to touch.

Project	Address	Before	After	Effect
`0x001a`	`0x1000026e`	`32`	`63`	`movs r1, #50` -> `#99` , prints `arithmetic_operator: 99`

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Toggle the lock off so editing is enabled.
3. Go to `0x1000026e` and change the byte `32` to `63` .
4. Return to the linear view, right-click the function -> `Reanalyze` .

Option B — Python console:

```
bv.write(0x1000026e, b"\x63")
print(hex(bv.read(0x1000026e, 1)[0]))  # -> 0x63
```

After reanalysis the instruction reads `movs r1, #99` .

Step 18b: Patch 2 — change the DHT11 scale from `0.1f` to `5.0f`

The DHT11 driver converts the integer and decimal bytes to a float and scales the decimal part:

```
humidity    = data[0] + data[1] * 0.1f
temperature = data[2] + data[3] * 0.1f
```

`dht11_read` computes both with a fused multiply-add against a single `0.1f` constant. Look at the end of `dht11_read` :

```
100003ee:       vmov    s15, r6
100003f2:       vcvt.f32.s32    s12, s15
100003f6:       vmov    s15, r1
100003fa:       vcvt.f32.s32    s14, s15
100003fe:       vmov    s15, r0
10000402:       vcvt.f32.s32    s13, s15
10000406:       vmov    s15, r2
1000040a:       vldr    s11, [pc, #44]  @ 10000438 <dht11_read+0x144>
1000040e:       vcvt.f32.s32    s15, s15
10000412:       movs    r0, #1
10000414:       vfma.f32        s14, s12, s11
10000418:       vfma.f32        s15, s13, s11
1000041c:       vstr    s14, [r5]
10000420:       vstr    s15, [fp]
```

The constant `s11` is loaded by the `vldr s11, [pc, #44]` at `0x1000040a` , which reads the literal-pool word at `0x10000438` . That word is `0x3dcccccd` (float `0.1`), stored little-endian as `cd cc cc 3d` . Change it to `0x40a00000` (float `5.0`), bytes `00 00 a0 40` :

Project	Address	Before	After	Effect
`0x001a`	`0x10000438`	`cd cc cc 3d`	`00 00 a0 40`	`0.1f` -> `5.0f` ; the humidity/temperature decimal parts read ~50x larger

```
+-------------------------------------------------------------------+
|  IEEE-754 little-endian change                                    |
|                                                                   |
|  0.1f = 0x3dccccd  ->  bytes  cd cc cc 3d                         |
|  5.0f = 0x40a00000  ->  bytes  00 00 a0 40                        |
|                                                                   |
|  humidity    = int + (decimal * 0.1f)   becomes                   |
|  humidity    = int + (decimal * 5.0f)                             |
|                                                                   |
+-------------------------------------------------------------------+
```

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x10000438` and change `cd cc cc 3d` to `00 00 a0 40`.
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x10000438, bytes.fromhex("0000a040"))
print(bv.read(0x10000438, 4).hex())  # -> 0000a040
```

> **The scale is applied to both humidity and temperature.** The two `vfma.f32` instructions both read `s11`, so patching the one constant changes both readings, not just temperature.

Step 18c: Patch 3 — rename the printed label

The string `"arithmetic_operator: %d\r\n"` starts at `0x100038e8`. Its first 26 bytes are:

```
61 72 69 74 68 6d 65 74 69 63 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00
 a  r  i  t  h  m  e  t  i  c  _  o  p  e  r  a  t  o  r  :     %  d \r \n \0
```

We replace it with `"hacked_operator: %d\r\n\0"`, which is **four bytes shorter**, and pad the tail with NULs:

```
68 61 63 6b 65 64 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00 00 00 00 00
 h  a  c  k  e  d  _  o  p  e  r  a  t  o  r  :     %  d \r \n \0 <pad>
```

The `\0` at offset 21 terminates the string, so `printf` stops there and the four padding bytes are never read. The next format string (`"increment_operator: …"`) begins at `0x10003904`, which is 28 bytes past `0x100038e8`, so our 26-byte write cannot reach it.

Project	Address	Before	After	Effect
`0x001a`	`0x100038e8`	`61 72 69 74 68 6d 65 74 69 63 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00`	`68 61 63 6b 65 64 5f 6f 70 65 72 61 74 6f 72 3a 20 25 64 0d 0a 00 00 00 00 00`	prints `hacked_operator:` instead of `arithmetic_operator:`

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x100038e8` and replace the 26 bytes above.
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x100038e8, b"hacked_operator: %d\r\n\x00\x00\x00\x00")
print(bv.read(0x100038e8, 26))  # -> b'hacked_operator: %d\r\n\x00\x00\x00\x00'
```

> **Keep the NUL.** `printf` walks the format string until it hits `\0`. If you omit it, the printer runs on into the padding and then into the next string, producing garbage. The four padding bytes guarantee the terminator.

Step 19: Export the patched `.bin`

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size come from the view itself
out = os.path.join(os.path.join(root, "0x001a_operators", "build"), "0x001a_operators-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 17484 /.../build/0x001a_operators-h.bin
```

Where the two numbers come from — nothing is hardcoded:

- `seg.start` is the image base IDA loaded the `.bin` at (`0x10000000`), the same value you pass to `uf2conv --base`.
- `seg.data_length` is the segment's size in the file (`0x444c` = 17484). Exactly one segment carries data (the image); every peripheral and synthetic segment has `data_length == 0`, so `next(...)` picks the image.
- Reading `seg.start` for `seg.data_length` bytes therefore grabs exactly the image.

Two gotchas this avoids:

- **No relative path.** IDA's Python console runs with a read-only working directory (inside the app bundle), so `open("0x001a_operators-h.bin", "wb")` fails with `OSError: [Errno 30] Read-only file system`. `root` (from `~/.embedded-hacking-repo`, Step 3) is the repo, so the file is written into the project's `build/` — no machine-specific path and no database needed.
- **Read the image, not the whole view.** `bv.read(bv.start, bv.length)` spans the entire mapped range, which is not the image. The segment's `data_length` is the image size.

A different size means you exported a partial view.

Step 20: Convert to UF2

Run from the project directory:

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x001a_operators-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x001a_operators-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

> **Or convert from the IDA console** — it is a normal Python interpreter, so you never have to leave the app. `chdir` to a writable directory first (the default one is read-only), then run the script:
>
> ```
> import os, sys, runpy
> os.chdir(os.path.join(root, "0x001a_operators", "build"))   # the project build dir (writable)
> sys.argv = ["uf2conv.py", "0x001a_operators-h.bin",
>             "--base", "0x10000000", "--family", "0xe48bff59", "--output", "hacked.uf2"]
> runpy.run_path("../../uf2conv.py", run_name="__main__")   # path to your uf2conv.py
> ```
>
> This writes `hacked.uf2` next to the `.bin`, ready to drag onto the Pico.

Static stopping point

The patched image has been exported and converted to UF2. This IDA guide ends at static verification; it does not connect to, flash, or debug a live target.

Glossary

Term	Definition
AAPCS	ARM Architecture Procedure Call Standard — `r0`-`r3` for the first four arguments, `r0` for the return value
Arithmetic operator	C operators for math (`+`, `-`, `*`, `/`, `%`); here `x * y` folds to `#50`
Assignment operator	Compound assignment (`+=`, `-=`, …); here `x += 5` folds to `#10`
Bitwise operator	Operators on individual bits (`<<`, `>>`, `&`, `|`, `^`); here `x << 1` folds to `#10`
`.bss`	Section for uninitialized (or zero-initialized) static/global variables; `dht_pin` lives here
`.data`	Section for initialized static/global variables; copied from flash to SRAM at boot
DHT11	Low-cost single-wire digital humidity and temperature sensor
`.elf`	Linked image with the symbol table; the ground truth for addresses and names
Fused multiply-add	`vfma.f32 Sd, Sn, Sm` computes `Sd = Sd + (Sn * Sm)` in one operation
Immediate value	A constant embedded directly in an instruction, not fetched from memory
Increment operator	`x++` (post) or `++x` (pre); here `x++` returns the old value, `5`
IEEE-754	Standard for floating-point representation; `0.1f` is `0x3dcccccd`
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative `ldr`
Logical operator	Operators combining conditions (`&&`, `||`, `!`); here `false`
`movs`	16-bit Thumb move that loads an 8-bit immediate (0-255); immediate is the first byte
`movw`	32-bit Thumb-2 "move wide" that loads a 16-bit immediate; low imm8 is the third byte

Term	Definition
PCF8574	An I2C I/O expander; not used this week (that was the LCD, Week 7)
Relational operator	Comparison operators (`<` , `>` , `==` , `!=`); here `5 > 10` is `false`
`.rodata`	Read-only section for constants and string literals; stays in flash
SIO	Single-cycle I/O block; `gpio_set_dir` / `gpio_put` write it with `mcrr` , `gpio_get` reads it
SVD	System View Description file — register names for a peripheral; the RP2350 one is at `WEEK04/rp2350.svd`
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
TIMER0	RP2350 timer block at `0x400b0000` ; `time_us_32` reads its `TIMERAWL` register (`+0x28`)
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Verify the static behavior from the recovered instructions.c `symbols), then patch the bytes – the` movs ` immediate (` 0x32 -> 0x63 `), the DHT11 float word (` 0.1f -> 5.0f `), and the format string (` arithmetic_operator: `->` hacked_operator: `) — and export the patched artifact.